

Relational Algebra, Functional Dependencies, and Normalization

Exam cheatsheet with compute methods and solved normalization exercise

This sheet is designed for the two exam skills that require computation: writing relational algebra expressions and analyzing a relation with functional dependencies. The method is always the same: identify schemas, compute closures, find keys, test normal forms, and decompose only when a dependency violates the target normal form.

Relational Algebra: Objects And Notation

Relational algebra manipulates relations and returns relations. In the course notation, $R[A, B]$ is projection, $R:P$ is selection, $R * S$ is natural join, $R(P)*S$ is theta join, and $R - S$ is set difference. The alternative textbook notation is also shown because exam statements may accept either style.

Operation	Notation	Meaning	Schema rule
Projection	$R[A, B]$ or $\pi_{A, B}(R)$	Keep only attributes A, B; duplicate tuples disappear because relations are sets.	Result has attributes A, B.
Selection	$R:P$ or $\sigma_P(R)$	Keep tuples of R satisfying predicate P.	Same attributes as R.
Rename	$T(X, Y) \leftarrow R[A, B]$	Give a name and possibly new attribute names to a result.	Use before self-joins or set operations with different names.
Cartesian product	$R \times S$	All pairs of tuples from R and S.	All attributes from both sides; qualify duplicates.
Theta join	$R(P)*S$	Cartesian product filtered by predicate P.	All attributes from both sides unless later projected.
Natural join	$R * S$	Equality join on all attributes with the same names, with duplicate join columns kept once.	Union of attributes of both relations.
Union	$R \text{ union } S$	Tuples in either relation.	Inputs must be union-compatible.
Intersection	$R \text{ intersect } S$	Tuples in both relations.	Inputs must be union-compatible.
Difference	$R - S$	Tuples in R but not in S.	Inputs must be union-compatible.
Division	R / S	Values in R that match all tuples of S.	If $R(X, Y)$ and $S(Y)$, result has X.

Union-compatible means the two input relations have the same number of attributes and corresponding domains are comparable. If the attribute names differ, rename the result or the operands explicitly.

Relational Algebra Compute Rules

Situation	Method that works
Simple filter and attributes	First select rows, then project columns: $R:\text{condition}[\text{attributes}]$. If you project first, the condition may no longer have the attributes it needs.
Join two tables	Join on keys, then select extra conditions, then project the requested attributes. Use theta join if the join columns have different names.
Same relation used twice	Rename two copies first, such as $R1(a, b) \leftarrow R[A, B]$ and $R2(c, d) \leftarrow R[A, B]$, then join R1 and R2.
“Never”, “none”, “not rated”, “without”	Compute all candidates, compute the bad candidates, subtract: $All - Bad$. Do not use $!=$ joins for absence.
“More than once”	Self-join two renamed copies and require same target but different witness, then project the target.
“Maximum” or “minimum” without aggregation	Find objects beaten by another object, then subtract from all objects. For maximum, remove objects with a strictly larger competitor.
“All”	Use division if allowed. Otherwise use double negation: candidate minus candidates missing at least one required item.
Aggregation request	Basic relational algebra in the slides has no COUNT, AVG, MAX, or GROUP BY. Write SQL, or state that it requires extended relational algebra.

English To Relational Algebra Patterns

English request	Relational algebra skeleton
Titles of books wished by Peter	$(Reader:Name='Peter' * Wishlist * Book)[Title]$, assuming common attributes or using theta joins if names differ.
Books in wishlist but never rated	$Wishlist[ISBN] - Rating[ISBN]$. This is the canonical absence pattern.
Books rated more than once	$R1(reader1, isbn, rating1) \leftarrow Rating; R2(reader2, isbn, rating2) \leftarrow Rating; ((R1(R1.isbn=R2.isbn \text{ and } R1.reader1 \neq R2.reader2)*R2)[isbn])$.
Actors who played in same movie as Woody Allen	Find Woody Allen's film ids, join with Role on film id, then join with Person on actor id, then project names.
Actors who never played in Woody Allen movies	All actors minus actors in films directed by Woody Allen. The subtraction must be between compatible relations, usually actor ids.
People who are both actors and directors	$Role[pid] \text{ intersect } Director[pid]$, then join with Person if names are requested.
People who played in all movies directed by Woody Allen	Let $WA(fid)$ be Woody Allen films and $Played(pid, fid)$ be actor-film pairs. The result is $Played / WA$, joined with Person.
Youngest or highest rank without aggregation	Create two copies; compute objects that are beaten by another; subtract them from all candidates.

Division Without Magic

Division answers “for all” questions. If $R(X, Y)$ says candidate X is linked to item Y, and $S(Y)$ is the set of required items, then R / S returns candidates linked to every item in S.

Line	Construction for $R(X, Y) / S(Y)$
All pairs	$All \leftarrow R[X] \times S$ gives every candidate-required-item pair that should exist.

Line	Construction for R(X,Y) / S(Y)
Missing pairs	Missing <- All - R gives candidate-item pairs that are required but absent.
Bad candidates	Bad <- Missing[X] gives candidates missing at least one required item.
Good candidates	Result <- R[X] - Bad gives candidates linked to all required items.

This expansion is useful when division notation is not allowed, or when you want to prove your answer is correct.

Functional Dependencies

A functional dependency $X \rightarrow Y$ on relation R means that any two tuples equal on attributes X must also be equal on attributes Y. Read it as “X determines Y”. It is a constraint on all legal instances of the relation, not just a coincidence in one table unless the exercise explicitly asks about a given instance.

Term	Definition and exam use
Trivial FD	$X \rightarrow Y$ is trivial when Y is already included in X, for example $AB \rightarrow A$. Trivial FDs never violate normal forms.
Non-trivial FD	$X \rightarrow Y$ is non-trivial when Y contains at least one attribute not already in X. These are the FDs tested for BCNF and 3NF.
Closure of attributes	X^+ is the set of all attributes determined by X under a set of FDs F. Use it to find keys and implied FDs.
Closure of FDs	F^+ is the set of all FDs implied by F. In practice, compute X^+ for relevant left sides instead of listing all of F^+ .
Superkey	X is a superkey of relation R(U) when $X^+ = U$. It determines all attributes.
Candidate key	X is a minimal superkey: $X^+ = U$ and no proper subset of X is a superkey.
Prime attribute	An attribute that belongs to at least one candidate key.
Non-prime attribute	An attribute that belongs to no candidate key. These are important for 2NF and 3NF.

Turning English Constraints Into FDs

An FD expresses uniqueness of a right-hand-side value for a fixed left-hand-side value. The sentence “for each X, there is at most one Y” becomes $X \rightarrow Y$. If the sentence needs counting, ordering, dates overlapping, or sets of values, it is usually not expressible as a simple FD.

Wording pattern	FD form	Reason
“An employee works in one department.”	Employee $\rightarrow$ Department	For a fixed employee, there is one department.
“A department is in one building.”	Department $\rightarrow$ Building	For a fixed department, there is one building.
“A driver cannot work on two lines on the same day.”	Driver, Day $\rightarrow$ Line	For a fixed driver and day, there is at most one line.
“A bus line is driven by a single driver.”	Line $\rightarrow$ Driver	For a fixed line, there is one driver. If this is meant per day, use Line, Day $\rightarrow$ Driver.
“A client can rent at most two places.”	Not a simple FD	FDs express one value, not “at most two” values. Use a separate cardinality constraint.
“A place cannot be rented twice at overlapping dates.”	Not a simple FD	This is temporal overlap, not equality of attributes inside two tuples.
“A line is always driven by the same drivers.”	Usually not a simple FD	If several drivers form a set, FDs cannot determine a set-valued attribute in 1NF. If the intended meaning is exactly one driver, then Line $\rightarrow$ Driver.

Armstrong Axioms And Practical Inference

You rarely need to name every axiom during an exam, but you must be able to justify derived dependencies. The closure algorithm is just Armstrong’s axioms executed mechanically.

Rule	Form	Use
Reflexivity	If Y subset X, then $X \rightarrow Y$.	Explains trivial dependencies such as $AB \rightarrow A$.
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$.	Allows adding the same context to both sides.
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.	Main rule for chained dependencies.
Decomposition	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.	Lets you test one RHS attribute at a time.
Union	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.	Lets you combine dependencies with the same determinant.
Pseudo-transitivity	If $X \rightarrow Y$ and $YW \rightarrow Z$, then $XW \rightarrow Z$.	Useful in longer derivations, but closure usually replaces it.

Attribute Closure Algorithm

To compute X^+ under F, start with exactly the attributes in X. Scan the dependencies. Whenever a dependency has its whole left side already inside the closure, add its right side. Repeat until a full scan adds nothing.

Step	Action
1	$X^+ \leftarrow X$.
2	For every FD $L \rightarrow R$ in F, if L subset X^+ , then replace X^+ by $X^+ \cup R$.
3	Repeat Step 2 until X^+ stops changing.
4	If X^+ contains all attributes of the relation, X is a superkey.

To derive all implied non-trivial dependencies over a small schema, compute X^+ for each non-empty subset X of attributes. Every attribute A in $X^+ - X$ gives an implied FD $X \rightarrow A$. For compact answers, remove dependencies that are obvious consequences of smaller left sides when the question asks for a minimal explanation rather than all of F^+ .

Candidate Key Algorithm

The fastest key method is to use the right-hand side shortcut first, then closure.

Step	Action
1	Let U be all attributes of the relation. Mark every attribute that never appears on the right-hand side of any FD. Each such attribute must be in every candidate key, because no dependency can derive it.
2	Start with the mandatory attributes M . Compute M^+ . If $M^+ = U$, then M is the only possible minimal key unless some mandatory attribute is redundant, which cannot happen if it never appears on a RHS.
3	If M^+ is not all attributes, add combinations of the remaining attributes, smallest combinations first, and compute closure.
4	Whenever a set determines all attributes, test minimality by removing one attribute at a time. If every removal fails, it is a candidate key.
5	Do not list non-minimal supersets as keys. Supersets of keys are superkeys, not candidate keys.

Normal Forms

Always find candidate keys before testing normal forms. Without keys, “prime attribute”, “partial dependency”, “transitive dependency”, and “superkey determinant” are unknown.

Form	Definition	Exam test
1NF	All attributes have atomic values. In this course, ordinary relations are assumed to be in 1NF.	No sets, lists, or repeating groups inside one cell.
2NF	1NF and every non-prime attribute fully depends on every candidate key.	Violation exists when a proper subset of a composite key determines a non-prime attribute. If all candidate keys are single attributes, 2NF is automatic.
3NF	For every non-trivial FD $X \rightarrow A$, either X is a superkey or A is prime. This is the compact test equivalent to the course’s non-transitive-dependency rule.	Check every relevant FD with one RHS attribute. Non-superkey determinant plus non-prime RHS violates 3NF.
BCNF	For every non-trivial FD $X \rightarrow A$, X must be a superkey.	Stricter than 3NF. Any non-key determinant violates BCNF, even if the RHS is prime.

The implication chain is $BCNF \Rightarrow 3NF \Rightarrow 2NF \Rightarrow 1NF$. If a relation is not 3NF, it cannot be BCNF. If a relation is BCNF, it is automatically 3NF and 2NF.

Normal Form Decision Table

FD $X \rightarrow A$	Status	Consequence
X is a superkey	Good for BCNF and 3NF	The dependency is controlled by a key, so it does not create the redundancy BCNF targets.
X is not a superkey and A is prime	Good for 3NF, bad for BCNF	This is the classic case where 3NF allows something BCNF rejects.
X is not a superkey and A is non-prime	Bad for 3NF and BCNF	The relation is at best 2NF, assuming no partial dependency violation.
X is a proper subset of a composite candidate key and A is non-prime	Bad for 2NF	The relation is only 1NF.

BCNF Decomposition Algorithm

BCNF decomposition removes a violating dependency by splitting the relation. It guarantees a lossless join at each split, but it may lose dependency preservation.

Step	Action
1	Start with the original relation $R(U)$ and its FDs F . Compute candidate keys.
2	Find a relation Q in the current decomposition that is not BCNF. This means there is a non-trivial FD $X \rightarrow Y$ holding in Q where X is not a superkey of Q .
3	Replace $Q(U_q)$ by $Q_1(X \cup Y)$ and $Q_2(X \cup (U_q - Y))$. Equivalently, keep X in both relations and move the attributes determined by X into their own relation.
4	Project the relevant FDs onto Q_1 and Q_2 , then test each new relation for BCNF.
5	Repeat until every relation is BCNF.

For a binary decomposition into R_1 and R_2 , the join is lossless when $(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$ follows from the dependencies. In the algorithm above, the common attributes are X , and $X \rightarrow Y$ makes the split lossless.

Projecting FDs After Decomposition

When a relation is decomposed, each new relation only keeps dependencies that mention attributes still present in that relation. The safe method is closure-based: for every subset X of the new relation’s attributes, compute X^+ using the original dependencies, then intersect with the new relation’s attributes.

Step	Action
1	Let the decomposed relation be $Q(V)$, where V is its set of attributes.
2	For each relevant subset $X \subseteq V$, compute X^+ with the original FD set.
3	Keep only attributes from V : the projected closure is $X^+ \cap V$.
4	Every attribute A in $(X^+ \cap V) - X$ gives a projected FD $X \rightarrow A$ on Q .
5	Use these projected FDs to test whether Q is BCNF or 3NF.

In small exams, you usually do not need to list every projected FD. It is enough to list the dependencies needed to show each decomposed relation is in BCNF, such as $C \rightarrow D$ for $R_1(C, D)$ and $A \rightarrow B, C$ for $R_2(A, B, C)$.

Common Normalization Mistakes

Mistake	Correction
Testing BCNF using only listed FDs	Normal forms are defined using all dependencies implied by the FDs. In simple exams, testing listed dependencies plus obvious derived dependencies is usually enough, but remember that implied violations count.
Calling every superkey a key	A key is minimal. AB is not a candidate key if A alone already determines all attributes.
Forgetting prime attributes	3NF permits a non-superkey determinant only when the RHS attribute is prime. BCNF does not.
Removing redundant FDs before checking key preservation	A dependency redundant in the original set may become necessary after other dependencies are removed.
Assuming BCNF preserves all dependencies	BCNF decomposition is lossless, but it can lose dependencies. If dependency preservation is required, say so and check it separately.
Using != to express absence in relational algebra	Absence is almost always set difference, not inequality join.

Example Exam Exercise 2: Subject

Consider the relation schema $R(A, B, C, D)$ with the following functional dependencies:

$F = \{ A \rightarrow B, A \rightarrow C, C \rightarrow D, A \rightarrow D, B \rightarrow D \}$.

The questions are as follows.

Part	Question
a	Compute the keys of the relation R under the functional dependencies F. Justify your answer.
b	Test the normal form of the relation R: is it in BCNF, 3NF, or 2NF? Justify your answer.
c	If necessary, decompose the relation R into a set of relations that are in BCNF. Detail your steps.
d	Which functional dependencies need to be removed in F so that R is BCNF and has the same key?

Example Exam Exercise 2: Fully Solved Answer

Part a: Keys

The right-hand sides of the dependencies are B, C, and D. Attribute A never appears on the right-hand side. Therefore every candidate key must contain A, because there is no way to derive A from other attributes.

Closure	Computation
A^+	Start with $\{A\}$. From $A \rightarrow B$, add B. From $A \rightarrow C$, add C. From $A \rightarrow D$, add D. Therefore $A^+ = \{A, B, C, D\}$.

Since A^+ contains all attributes of R, A is a superkey. Since A has no proper non-empty subset, it is minimal. Since every key must contain A, no other candidate key exists. The only candidate key is therefore A. The only prime attribute is A; attributes B, C, and D are non-prime.

Part b: Normal Form

The relation is in 1NF because it is a relation schema with atomic attributes. The only candidate key is the single attribute A, so there cannot be a partial dependency on a proper subset of a composite key. Therefore the relation is in 2NF.

FD	Is LHS a superkey?	RHS prime?	Normal-form effect
$A \rightarrow B$	Yes	No	OK for BCNF because A is a key.
$A \rightarrow C$	Yes	No	OK for BCNF because A is a key.
$A \rightarrow D$	Yes	No	OK for BCNF because A is a key.
$C \rightarrow D$	No, because $C^+ = \{C, D\}$	No	Violates 3NF and BCNF.
$B \rightarrow D$	No, because $B^+ = \{B, D\}$	No	Violates 3NF and BCNF.

Because $C \rightarrow D$ and $B \rightarrow D$ have non-superkey determinants and non-prime right-hand side D, the relation is not in 3NF. Since BCNF is stricter than 3NF, the relation is not in BCNF either. The highest normal form satisfied by R is 2NF.

Part c: BCNF Decomposition

We must decompose because the relation is not BCNF. Choose the violating dependency $C \rightarrow D$. The determinant is C, and C is not a superkey of R.

Step	Decomposition
1	Use the BCNF split for $C \rightarrow D$: create $R_1(C, D)$ and $R_2(A, B, C)$. The common attribute is C, and $C \rightarrow D$, so the decomposition is lossless.
2	Check $R_1(C, D)$. The projected dependency is $C \rightarrow D$. In R_1 , $C^+ = \{C, D\}$, so C is a key. Therefore R_1 is BCNF.
3	Check $R_2(A, B, C)$. The projected dependencies include $A \rightarrow B$ and $A \rightarrow C$, so $A^+ = \{A, B, C\}$ in R_2 . Thus A is a key of R_2 . The non-trivial dependencies in R_2 have determinant A, so R_2 is BCNF.

One valid BCNF decomposition is therefore:

$R_1(C, D)$
 $R_2(A, B, C)$

This decomposition is lossless. It does not preserve the original dependency $B \rightarrow D$, because no resulting relation contains both B and D. This is acceptable for the standard BCNF decomposition algorithm unless dependency preservation is explicitly required.

An alternative valid BCNF decomposition can be obtained by first choosing $B \rightarrow D$, giving $R_1(B, D)$ and $R_2(A, B, C)$. Different choices of violating dependencies can lead to different valid BCNF decompositions.

If the examiner also wants dependency preservation, add the missing dependency relation. For this example, the decomposition $R_{ABC}(A, B, C)$, $R_{CD}(C, D)$, and $R_{BD}(B, D)$ is still BCNF and lossless because R_{ABC} with R_{CD} was already lossless, and adding another projection cannot create extra tuples. It also preserves $B \rightarrow D$. The standard BCNF algorithm does not require this extra relation unless dependency preservation is requested.

Part d: Removing FDs So The Original Relation Is BCNF With The Same Key

The key must remain A, so the remaining dependencies must still make $A^+ = \{A, B, C, D\}$. The dependencies that violate BCNF are exactly the dependencies whose left side is not a superkey: $C \rightarrow D$ and $B \rightarrow D$.

Remove:

$C \rightarrow D$
 $B \rightarrow D$

Keep:

$\{ A \rightarrow B, A \rightarrow C, A \rightarrow D \}$

Under the remaining set, $A^+ = \{A, B, C, D\}$, so A is still the candidate key. Every remaining non-trivial dependency has determinant A, which is a key. Therefore the original schema $R(A, B, C, D)$ is BCNF under the reduced dependency set and has the same key as before.

One-Page Method For Any Normalization Exercise

Order	Action
1	Rewrite all FDs with one attribute on the right when testing normal forms, such as $A \rightarrow BC$ becoming $A \rightarrow B$ and $A \rightarrow C$.
2	Compute RHS attributes. Any attribute missing from all RHSs must be in every key.
3	Compute closures to find all candidate keys. Stop calling a set a key if a smaller subset is already a key.
4	Mark prime and non-prime attributes from the candidate keys.
5	Test 2NF by looking for proper subsets of composite keys determining non-prime attributes. If every key is atomic, 2NF is automatic.
6	Test 3NF using: for every non-trivial $X \rightarrow A$, X must be a superkey or A must be prime.
7	Test BCNF using: for every non-trivial $X \rightarrow A$, X must be a superkey.
8	If not BCNF, choose a violating FD $X \rightarrow Y$, split into XY and X plus the remaining attributes, then repeat.
9	Check losslessness using the intersection rule. Check dependency preservation only if the question asks or if you want to mention a limitation.

One-Page Method For Any Relational Algebra Exercise

Order	Action
1	Write the schemas and underline or mentally mark the join attributes.
2	Identify the output attributes requested by the question. Projection happens at the end unless an earlier projection simplifies the expression without removing needed attributes.
3	Translate filters into selections and relationship navigation into joins.
4	If the same relation must play two roles, create renamed copies before joining.
5	If the wording contains “not”, “never”, “none”, or “without”, build a set difference expression.
6	If the wording contains “all” or “every”, use division or the expanded division method.
7	If the wording contains “most”, “highest”, “average”, “number of”, or “count”, basic relational algebra is not enough unless extended aggregation is allowed. Use SQL for those parts or state the limitation.
8	Ensure both sides of union, intersection, and difference are union-compatible. Rename attributes if necessary.